

# API

## ¿Qué es una API?

El acrónimo **API** significa *Application Programming Interface* o (Interfaz de Programación de Aplicaciones).

Es un conjunto de reglas y protocolos que permite que dos aplicaciones se comuniquen entre sí.

Actúa como un contrato de comunicación: define cómo un sistema (el cliente) puede solicitar información o acciones a otro sistema (el servidor) y qué estructura tendrá la respuesta.

Las respuestas del servidor suelen venir en formato JSON (Java Script Object Notation), que es la misma estructura de los diccionarios en Python.

## Ejemplo intuitivo

Tu eres el cliente en un restaurante, no vas a la cocina (servidor) a pedir el alimento, lo haces mediante un intermediario (mesero / API).

Al igual que un mesero espera una compensación por traerte tus alimentos, la mayoría de API cobran por mantener los servidores corriendo, el mantenimiento de la banda de red, porque la información es exclusiva, limitar los usos para evitar saturaciones o ataques DDoS.

## Usos Principales de las APIs

**1. Conectar Aplicaciones** Permiten que sistemas totalmente independientes, desarrollados en diferentes lenguajes o tecnologías, compartan datos.

**2. Automatizar Procesos** Permiten disparar tareas repetitivas de forma programática sin intervención humana.

Por ejemplo de automatización: Cada vez que un cliente llena un formulario de registro en una landing page, una API puede conectar automáticamente ese evento para dar de alta al contacto, enviar un correo de bienvenida automatizado.

## Ejemplos de APIs Cotidianas

En nuestro día a día interactuamos constantemente con diferentes tipos de APIs. A continuación, se enlistan algunos escenarios comunes:

**APIs REST** Se usan en Aplicaciones web estándar como al entrar a un perfil de Spotify y que cargue las imágenes, lista de más populares, cuantas vistas tienen, etc:

**GraphQL** Se usan para solucionar el problema principal de las REST API (veremos mas adelante) (ej. Redes sociales, Tiktok, Netflix)

**gRPC Google Remote Procedure Call** Envía datos comprimidos entre muchos micro-servicios practicamente en tiempo real (ej. Uber o Didi para rastrear las ubicaciones).

**WebSockets** Permiten una comunicación **bidireccional** continua y en tiempo real sin necesidad de abrir y cerrar conexiones en cada petición (ej. WhatsApp, juegos multijugador).

**SOAP Simple Object Access Protocol** Protocolo estricto basado en **XML** que prioriza la seguridad y los contratos formales de datos (ej. Aplicaciones bancarias, sistemas de facturación gubernamentales).

## Endpoints y CRUD

Un **endpoint** es la dirección URL específica (por ejemplo, `https://api.tienda.com/v1/productos`) a la cual se realiza una petición para interactuar con un recurso en el servidor.

Para operar sobre estos endpoints, utilizamos operaciones CRUD (*Create, Read, Update, Delete*):

- **GET:** Se utiliza para **leer** o consultar información del servidor. (Ej: `lista_enfermedades`).
- **POST:** Se utiliza para **crear** un nuevo recurso en el servidor. (`clasificate_image`).
- **PUT:** Se utilizan para **actualizar** datos ya existentes. (Ej: Cambiar la contraseña o dirección de un usuario (**Datacamp**)).
- **DELETE:** Se utiliza para **borrar** o eliminar un recurso del servidor. (Ej: Dar de baja una cuenta de usuario).

## Limitaciones de la Arquitectura REST

- **Over-fetching:** El servidor devuelve más información de la que el cliente realmente necesitamos, haciendo que consumas antes tus llamadas (en algunas) o que tengas que limpiar mucho el formato de los datos.
- **Under-fetching:** Un endpoint no provee suficiente información, obligando al cliente a realizar múltiples peticiones a diferentes endpoints, gastando mas rapido las llamadas que tiene disponible.

## GraphQL: Evolución y Alternativa

Para solucionar las limitaciones de sobre-descarga y sub-descarga de REST, **GraphQL fue creado internamente por Meta en 2012** como un lenguaje de consulta para sus aplicaciones móviles. Años más tarde, **fue liberado y sacado al público en 2015**.

Permite al cliente especificar exactamente qué datos necesita mediante una sola consulta (*query*). El servidor procesa la petición y responde únicamente con la estructura exacta solicitada.